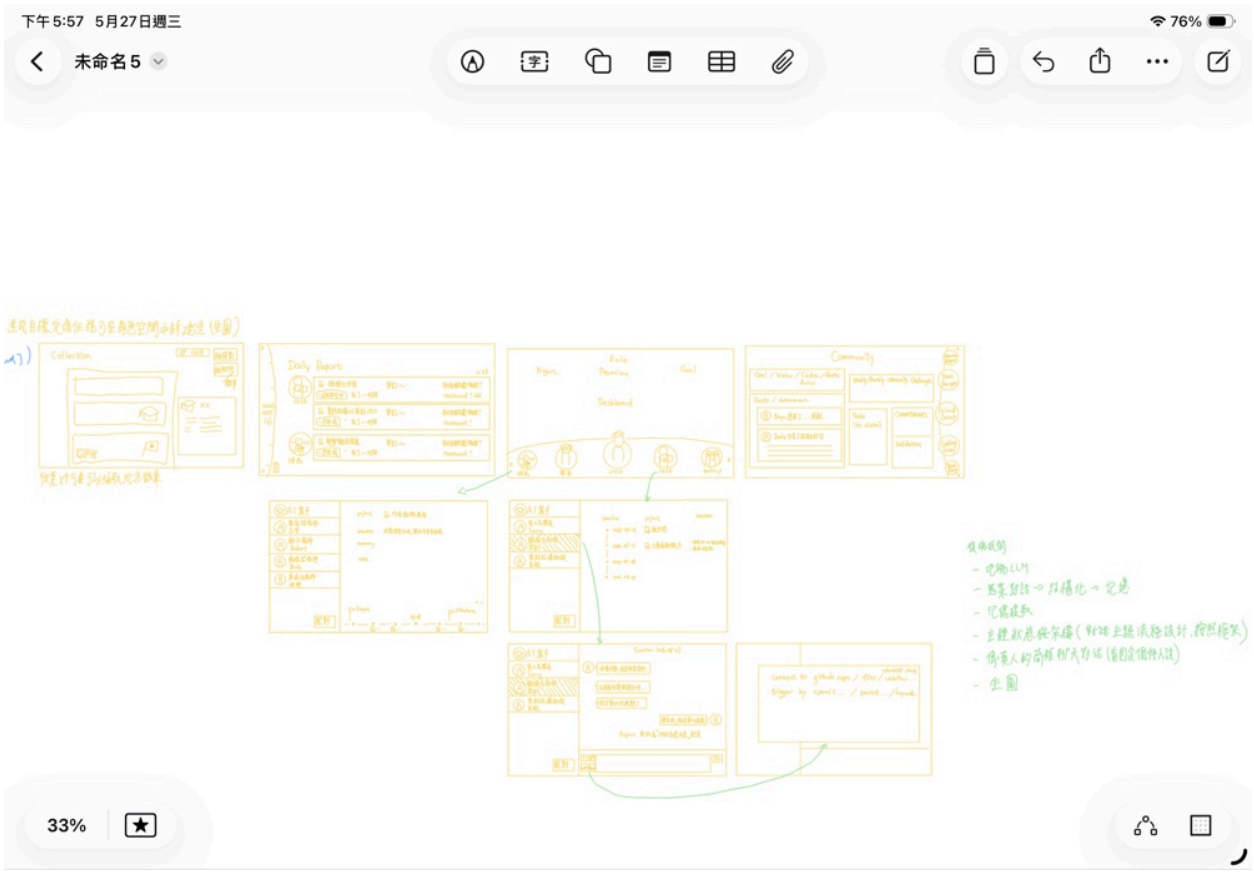


這是一張結合了「個人目標管理」、「AI 角色輔助」、「遊戲化機制（Gamification）」與「社群互動」的產品 UI/UX 線框圖（Wireframe）。這張圖具體將您上一張圖表中提到的「多巴胺刺激」、「社會情懷」與「成就感」轉化為實際的軟體介面架構。

以下為您將軟體整體的頁面架構與設計，完整轉譯成自然語言描述：

1. Collection（收藏與進度頁面，左上角）

此頁面是使用者成就與遊戲化機制的視覺展示中心。



這是一張展示「收藏與獎勵系統（Collection）」的 UI/UX 線框圖。這個頁面完美呼應了您在最一開始發想圖表中提到的「多巴胺刺激」、「收集」、「不確定性（盲盒）」以及「成就感」等核心心理學機制。

以下為您將此頁面的架構、內容、UI/UX 邏輯，以及實作所需的技術進行完整拆解描述：

一、頁面架構與內容描述

此頁面採用了經典的「主從式架構（Master-Detail Pattern）」，將左側的收藏清單與右側的詳細資訊面板並列，並在頂部整合了資源消耗機制。

1. 頂部資源與操作區 **(Header & Actions)**

- 頁面標題：左上角標示 `Collection`（收藏），確立此空間為使用者的成就展示櫃。
- 資源展示：右上角醒目地顯示 `XP 1234`，這是使用者透過 Daily Report 或 AI 任務互動所累積的「經驗值」或「貨幣」。
- 抽獎/兌換按鈕：XP 顯示區旁設有兩個具備盲盒機制的按鈕：
  - 抽背景（抽取系統或角色空間的自訂背景）
  - 抽物件 / 徽章（抽取虛擬裝飾品或成就徽章）

2. 左側：收藏展示櫃 **(Inventory / Collection List)**

- 架構：以垂直排列的長方形卡片（Slots）構成，展示已獲得的成就或物件。

- 內容範例：
    - 第一格：目前為空白，預留給未來的新收藏。
    - 第二格：帶有「學士帽」的圖示，可能代表某個階段性的學習成就（如前述角色的 UNI）。
    - 第三格：帶有 CPE 字樣與一面「旗幟」圖示。這具體示範了如何將硬核的學術里程碑（大學程式能力檢定），轉化為具備遊戲化收藏價值的虛擬資產。

3. 右側：詳細資訊面板 **(Detail View Panel)**

- 架構：一個方形的資訊卡片，當使用者點擊左側的某個收藏品時，此區塊會連動顯示細節。
- 內容：包含了被選取物件的圖示（如學士帽）、標題（XX 佔位符），以及下方數行橫線代表的「詳細描述」，預期會記錄該徽章的取得日期、背後的故事或解鎖條件。

4. 底部設計註解 **(Footer Note)**

- 文字：「收集 XP 可能 可以抽取紀念徽章」。這句話點出了整個系統的經濟循環（Economy Loop）：做任務 -> 拿 XP -> 消耗 XP 抽盲盒 -> 獲得徽章 -> 產生多巴胺與成就感。

二、UI / UX 互動設計邏輯

這個頁面是軟體中負責「留存率（Retention）」與「成癮機制」的核心：

- 不確定性的多巴胺回饋 **(Variable Ratio Schedule)**：抽背景 與 抽物件 的設計，運用了心理學中的「變動比例增強」。與固定給予獎勵相比，這種「不知道會抽到什麼」的盲盒機制，能最大化大腦中多巴胺的分泌，讓「累積 XP」這件枯燥的事變得充滿期待感。
- 低認知負擔的瀏覽體驗：主從式架構（左列表、右細節）讓使用者可以在不切換頁面的情況下，快速瀏覽並回味自己過去達成的種種挑戰，強化「掌控感」與對系統的「依賴性」。
- 身分認同的客製化：提供抽取「背景」與「物件」，不僅僅是收集，更能讓使用者自訂前述的「角色儀表板（Dashboard）」，滿足自我表達與炫耀的心理需求。

三、延伸技術建構需求

要實作這個具備遊戲化經濟與抽卡機制的頁面，後端邏輯的嚴謹性將大幅提升：

1. 前端開發技術 **(Frontend Architecture)**

- 動畫與微互動 **(Micro-interactions & Animations)**：抽卡體驗的靈魂在於「視覺回饋」。當按下 抽徽章 時，前端必須實作華麗的開箱動畫（例如使用 **Lottie** 動畫庫、CSS 粒子特效或 Framer Motion），營造出驚喜感。
- 狀態同步：前端需精準管理 XP 的狀態。抽卡扣除 XP 與獲得新徽章的畫面更新必須是原子化且同步的。
- 自適應網格佈局：左側的卡片列表可使用 CSS Grid 進行實作，確保在不同裝置上都能整齊排列。

2. 後端與資料庫技術 **(Backend & Database)**

- 交易安全性 **(Transaction ACID Properties)**：這是最重要的部分。扣除 XP 與寫入新徽章

到資料庫的過程，必須包裝在資料庫的「交易（Transaction）」中。若過程中發生錯誤，必須全部 Rollback，絕對不能發生「XP 扣了卻沒拿到徽章」或「利用並發請求（Race Condition）無限刷徽章」的漏洞。

- 亂數與機率演算法 **(RNG & Weighted Gacha System)**：抽卡的機率計算必須完全在後端執行，絕對不能寫在前端。後端需要實作一套帶有權重（Weights）的隨機數生成器，定義好普通、稀有、史詩級徽章的掉落機率。
- 資料表設計：
  - `Users` 表：增加 `xp_balance` (整數型別) 欄位。
  - `Items_Dictionary` 表：存放所有可抽取背景、徽章的 Metadata（圖片 URL、稀有度、描述）。
  - `User_Collections` 表：關聯表，紀錄哪個 User 擁有了哪個 Item，以及取得的 Timestamp。

3. AI 資產生成 (AI-Generated Assets) - 進階延伸

呼應您之前提到的「生圖」技術，這裡的徽章或背景(需立體)不一定要由設計師手繪。可以整合 **Stable Diffusion** 或 **Midjourney API**。當使用者完成一個特定專案（如：完成一個 RL 機器手臂抓取任務）並觸發抽卡時，系統能根據該任務的 Context Prompt，即時生成一張「獨一無二的專屬紀念徽章」，將個人化體驗推向極致。

2. Daily Report（日報頁面，中上左）

這是一個定期檢視進度的儀表板，帶有強烈的回顧與自我反思意味。



這是一張聚焦於「日報（Daily Report）」模組的 UI/UX 線框圖，將使用者的日常任務與學習歷程，透過時間軸與「角色（Role）」進行分類與回顧。

以下為您將此頁面的架構、內容、UI/UX 邏輯，以及實作所需的技術進行完整拆解描述：

一、頁面架構與內容描述

此頁面採用「左側時間軸」搭配「右側任務卡片」的佈局，並透過角色分組來呈現當日的行為紀錄。

1. 全域標題與獎勵機制 **(Header)**

- 左上角：標示大字標題 `Daily Report`（日報）。
- 右上角：標示 `+XP`，對應前述架構中「透過目標完成獲取經驗值」的遊戲化回饋機制。

2. 左側時間軸導覽區 **(Timeline Sidebar)**

- 內容包含：一條垂直的引導線，旁邊標註了具體的日期 `2026 MAY 16`。
- 底部圖示：時間軸最下方有一個類似日曆或手帳的圖示（帶有 `16` 字樣），作為視覺收尾或日期切換的錨點。
- 架構功能：提供明確的時間語境，使用者可以上下滑動來檢視不同日期的紀錄。

3. 主內容區：依角色分組的任務清單 **(Role-based Task Cards)**

任務紀錄並不只是流水帳，而是被嚴格歸類在不同的「角色/身分」之下。

- 群組 **A**：**CSIE**（資工）角色
  - 角色標示：左側有圓形頭像（手持筆電的小人），下方標註 `CSIE`。
  - 任務卡片 **1**（進行中）：
    - 標題與圖示：文件圖示搭配 `微積分作業`。
    - 狀態與耗時：標示 `正在進行中`（膠囊狀按鈕）與 `花了～時間`。
    - 反思紀錄：`學到了～`。
    - 檢視機制：右側提問 `有違背目標/偏誤？`（或與目標/承諾的對齊狀況），下方帶有 `reviewed? click` 的互動提示。
  - 任務卡片 **2**（已完成）：
    - 標題與圖示：文件圖示搭配 `資料結構AI筆記 CH2`。
    - 狀態與耗時：標示 `已完成`（膠囊狀按鈕）與 `花了～時間`。
    - 反思紀錄：`學到了～`。
    - 檢視機制：同樣帶有目標檢視與 `reviewed?` 提示。
- 群組 **B**：諮商 角色
  - 角色標示：左側有圓形頭像（帶有動作感的小人），下方標註 `諮商`。
  - 任務卡片 **3**（已完成）：
    - 標題與圖示：文件圖示搭配 `聊聊職涯發展`。
    - 狀態與耗時：標示 `已完成` 與 `花了～時間`。
    - 反思紀錄：`學到了～`。
    - 檢視機制：帶有目標檢視與 `reviewed?` 提示。

二、UI / UX 互動設計邏輯

此區塊的 UX 核心在於「歷程視覺化」與「引導深度反思」。

- 降噪與情境隔離（**Contextual Grouping**）：透過「角色」來分組任務（如：資工系的課業歸一類、職涯諮商歸另一類），能有效降低使用者在回顧時的認知干擾，幫助大腦快速切換不同的生活語境。
- 微摩擦力的反思機制（**Friction for Reflection**）：右側的 `reviewed? click` 與 `學到了～` 是設計上的巧思。系統不只是讓使用者「打勾完成」任務（這太過廉價），而是要求使用者進行點擊確認（Review），並思考這個任務是否「偏離/違背了當初設定的目標與承諾」。這種設計能有效提升任務完成後的「成就感」與「實在感」。
- 時間感與狀態感知：明確區分「正在進行中」與「已完成」，並記錄「花了多少時間」。這不僅能作為未來的數據分析基礎（例如分析進退步），也能帶給使用者掌控感。

三、延伸技術建構需求

要將此 Daily Report 頁面實作出來，並維持流暢的使用體驗，需要以下技術的支援：

1. 前端開發技術 **(Frontend Architecture)**

- 時間軸 **UI** 實作：左側的時間軸與右側卡片的對齊，強烈建議使用 CSS `Grid` 或 `Flexbox` 佈局。時間軸的垂直線可以透過 `border-left` 或偽元素（`::before`, `::after`）來繪製。
- 狀態管理與渲染 **(State Management)**：需要前端框架（如 React / Vue）來管理各個任務的

狀態（`status: 'in-progress' | 'completed'`）。當使用者點擊 `reviewed? click` 時，卡片的 UI 狀態應即時更新（例如改變背景顏色或加上核取記號）。
- 資料結構映射 **(Data Mapping)**：前端需要能處理巢狀資料結構的渲染。資料層級為 `Date -> Roles -> Array of Tasks`。

2. 後端與資料庫技術 **(Backend & Database)**

- 時間追蹤與紀錄儲存：
  - 針對「花了多少時間」，資料庫中的 `Task` 表單需要有 `started_at` 與 `ended_at` 的時間戳記（Timestamp）欄位，以便精算時間。
  - 若是結合您知識管理中常用的 Markdown 系統，後端也可能需要將這些結構化資料（JSON）定期編譯/同步成實體的 Markdown 日誌檔案（例如透過腳本整合到 Obsidian 的 Daily Note 中）。
- 關聯式查詢：API 查詢時，需要具備 `JOIN` 的能力。例如取得 `2026-05-16` 的資料時，必須一併拉取關聯的 `Role` 資訊與 `Task` 明細。

3. 系統連動與自動化 **(Integration & Automation)**

- **Review** 觸發機制：點擊 `reviewed? click` 時，除了更新資料庫狀態，也可設計為觸發（Trigger）Webhook。例如：按下 review 後，自動呼叫上一張圖提到的「AI 幫手」，讓 AI 根據您填寫的「學到了～」給予回饋，或是自動計算並發放 `+XP` 到您的帳號中。

3. Dashboard（角色與專案儀表板，正上方）

這是系統的核心中樞，整合了使用者的不同面向。



這是一張聚焦於系統「角色切換（Role-based Dashboard）」模組的細部 UI/UX 線框圖。這個設計具體化了個人知識與目標管理中，依照不同「身分」來建立專屬工作空間的概念。

以下為您將此頁面的架構、內容、UI/UX 邏輯，以及實作所需的技術進行完整拆解描述：

一、頁面架構與內容描述

此頁面採用「上、中、下」三個層次的佈局，將使用者的不同生活面向進行封裝與狀態隔離。

1. 頂部資訊區（**Context Header**）

- 內容包含：`Project`（專案）、`Role`（角色）、`Promises`（承諾）、`Goal`（目標）。
- 架構功能：作為當下狀態的「情境錨點」。當使用者在下方切換不同的角色時，這四個標籤的內容會動態更新，確保使用者隨時明確知道「我現在是誰」、「我在這個身分下的長期目標與短期承諾是什麼」。

2. 中間內容展示區（**Dashboard Body**）

- 內容包含：標註著大字的 `Dashboard`。
- 架構功能：這是一塊動態的畫布或資訊儀表板。預期會根據頂部定義的專案與目標，渲染出對應的待辦清單、數據圖表或微量 XP 進度。

3. 底部導覽與切換列（**Role Selector / Dock**）

- 內容包含：以橫向卷軸（Carousel）形式呈現的五個圓形頭像與身分標籤。從左至右分別為：
    1. `諮商`：搭配一個帶有動作感的人像圖示。
    2. `學者`：搭配一個戴眼鏡的人像圖示。
    3. **`UNI`**（大學生）：位於正中央，尺寸最大，並戴著學士帽圖示。
    4. **`CSIE`**（資工系）：搭配一個使用筆記型電腦的人像圖示。
    5. **`FAMILY`**（家庭）：搭配三個大小不一的人像，象徵家庭成員。
- 兩側控制元件：左右兩端帶有 ◁ 與 ▷ 箭頭，指示此區塊可左右滑動。

二、UI / UX 互動設計邏輯

此區塊的 UX 核心在於「降低認知負擔」與「建立身分邊界」。

- 動態置中焦點（**Cover Flow / Snap to Center**）：圖中的 `UNI` 角色位於正中央，且圖示比例明顯大於其他兩側的角色（如 `學者`、`CSIE`）。這暗示了一種常見的 UI 互動模式：當使用者左右滑動時，停留在中央的角色會被「放大」並判定為「目前選取的啟用狀態（Active State）」。
- 全域狀態切換（**Global State Management**）：底部的 Role Selector 是一個全域控制器。例如，當您將焦點從 `UNI` 滑動並切換至 `CSIE` 時，上方的 `Dashboard` 內容必須無縫切換，專注於資訊工程領域的相關任務；若切換到 `學者` 角色，儀表板則會重新渲染，顯示與國科會大專生計畫或文獻研究相關的進度與目標。
- 儀式感營造：透過具體的 Icon 與明確的標籤切換，幫助使用者在腦中建立「模式切換」的儀式感，減少不同領域專案間的上下文切換耗損（Context Switching Cost）。

三、延伸技術建構需求

要將此頁面的架構與流暢的 UX 實作出來，需要以下前後端與資料架構技術的支援：

1. 前端開發技術 **(Frontend Architecture)**

- 組件化框架 **(Component-Based Framework)**：建議使用 React、Vue 或 Svelte。這類頁面高度依賴組件的重複使用與狀態更新（例如根據 Role 重新渲染 Dashboard）。
- 全域狀態管理 **(State Management)**：需要如 Redux、Zustand (React) 或 Pinia (Vue) 來管理目前的「Active Role」。當底部 Dock 發生切換時，必須將 `currentRole` 狀態發送給全域，讓上方的 `Header` 與 `Dashboard` 監聽到變化並重新獲取資料。
- 流暢動畫與手勢操作 **(Animations & Gestures)**：
    - 底部 Carousel 的滑動、放大縮小效果（平滑過渡）是體驗的關鍵。
    - 在 Web 端可使用 **Framer Motion** 或 **GSAP** 來實作物理慣性滑動與中央放大的特效。
    - 若為行動裝置 APP，則需處理觸控滑動（Swipe）與對齊（Snap）的底層手勢庫。
- **UI** 樣式與排版：Tailwind CSS 等工具能快速切出 Flexbox 佈局，處理上中下的自適應高度分配。

2. 後端與資料庫技術 **(Backend & Database)**

- 關聯式與非關聯式資料設計：
    - 使用者表 (User) -> 角色表 (Roles) -> 專案表 (Projects) -> 目標表 (Goals)。這需

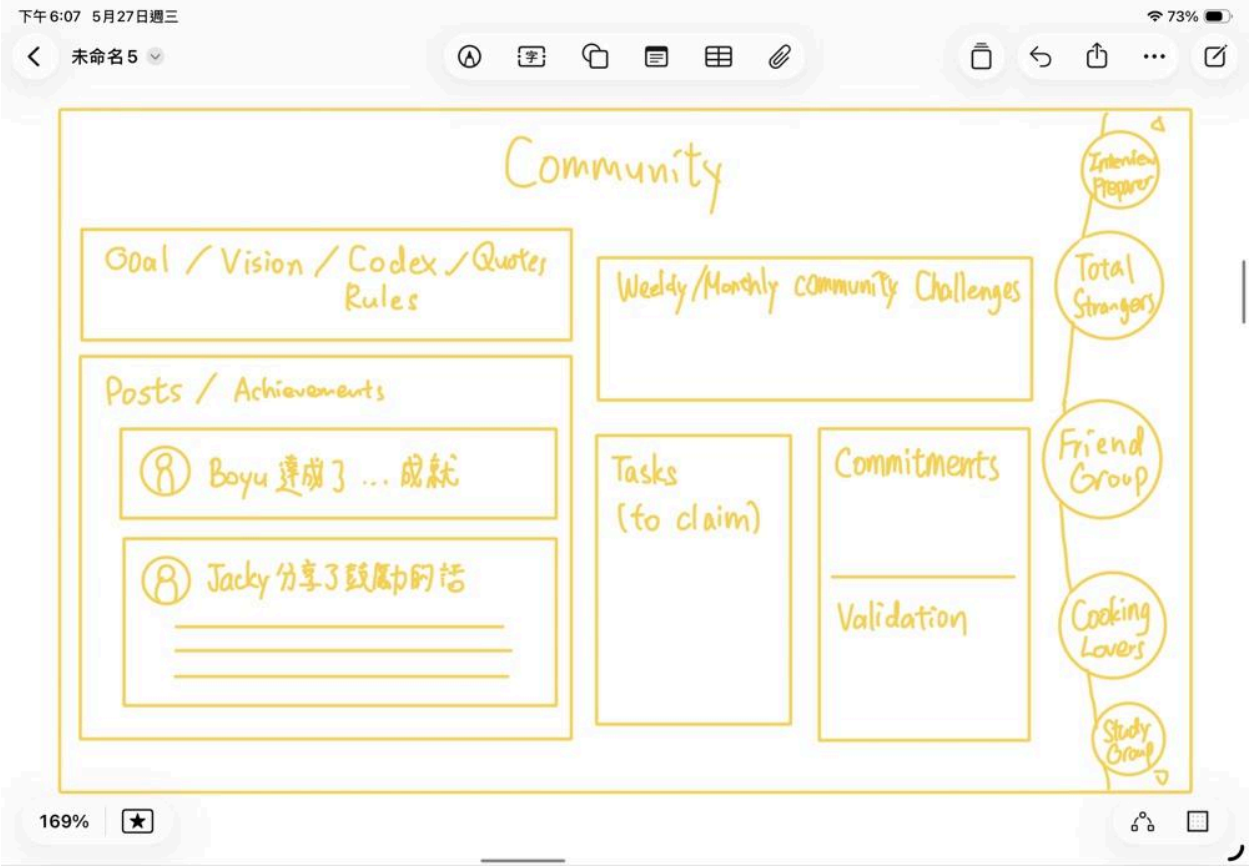
要明確的關聯式架構（如 PostgreSQL）。
    - 但 Dashboard 中的「小工具（Widgets）」或「筆記內容」可能結構較為多變，若結合您過去開發後端 API 或微服務的經驗，可設計一個彈性的 JSON 欄位或搭配 NoSQL（如 MongoDB）來儲存各個角色的儀表板配置。
- **API 設計 (RESTful 或 GraphQL)**：當前端切換角色時，需要呼叫 API 獲取新角色的資料。例如：`GET /api/users/me/dashboard?role=CSIE`。使用 GraphQL 可以更精準地只拉取切換後所需的畫面欄位，減少資料傳輸量。

3. 系統架構設計模式 (Design Patterns)

- 發布-訂閱模式 **(Pub/Sub)** 或 觀察者模式 **(Observer Pattern)**：在前端架構中，底部的 Selector 扮演發布者，當狀態改變時，通知訂閱了此狀態的上層組件進行重繪，確保資料流向單一且清晰，避免「義大利麵條式代碼（Spaghetti code）」。

4. Community（社群互動頁面，右上角）

此頁面將個人的成長與外部的「社會情懷」連結起來。



這是一張聚焦於「社群（Community）」模組的 UI/UX 線框圖。這個頁面將您先前構思圖表中的「社會情懷」、「公開承諾」與「互相依賴」等抽象心理學概念，轉化成了具體可互動的系統介面。

以下為您將此頁面的架構、內容、UI/UX 邏輯，以及實作所需的技術進行完整拆解描述：

一、頁面架構與內容描述

此頁面採用「區塊化儀表板（Bento/Grid Layout）」搭配「右側垂直導覽」的佈局，將社群互動的各種元素清晰劃分。

1. 全域標題 **(Header)**

- 內容：正上方標註大字標題 `Community`（社群），確立此頁面的核心語境。

2. 左半部：共識與動態資訊區 **(Context & Feed)**

- 社群共識區（左上）：
  - 標籤包含 `Goal / Vision / Codex / Quotes / Rules`（目標 / 願景 / 準則 / 名言 / 規範）。
  - 此區塊靜態展示當前社群的核心精神與行為規範，確保所有參與者具有共同的認知底線。
- 成就與貼文動態牆（左下）：
  - 標題為 `Posts / Achievements`。
  - 內部為條列式的動態卡片，範例包含：
    - 「`Boyu 達成了 ... 成就`」：系統自動推播或手動分享的里程碑。
    - 「`Jacky 分享了鼓勵的話`」：附帶多行文字線條，表示這是一則純文字的激勵貼文。

3. 中右半部：挑戰與行動承諾區 **(Actionable Items)**

- 定期挑戰區（右上）：
  - 標題為 `Weekly/Monthly community Challenges`（每週/每月社群挑戰），提供一個大型的空白區塊展示近期的共同任務。
- 任務認領與驗證區（右下，切分為兩塊）：
  - 左側為 `Tasks (to claim)`：列出目前社群中待領取、待解決的任務清單。
  - 右側為 `Commitments`（承諾）與 `Validation`（驗證）：兩者被一條水平線隔開，構成一個「承諾後需被驗證」的行為閉環。

4. 最右側：子社群切換導覽列 **(Sub-community Sidebar)**

- 架構：以一條垂直線串聯多個圓形按鈕，類似節點地圖的設計。
- 內容：由上而下標示了不同性質的社群分群：
  - `Interview Preparer`（面試準備群）
  - `Total Strangers`（完全陌生人）
  - `Friend Group`（朋友圈）
  - `Cooking Lovers`（烹飪同好）
  - `Study Group`（讀書會）

二、UI / UX 互動設計邏輯

此區塊的 UX 核心在於「利用社會語境建立習慣」與「精準控制社交邊界」。

- 同儕制約與正向激勵：`Posts / Achievements` 區塊將個人成就（如前一張圖的 Daily Report XP）廣播給學術或專案夥伴看見。看見如 Boyu 等熟悉夥伴達成成就，能觸發多巴胺中的「社交刺激」與「勝任感」渴望，進而帶動良性競爭。
- 將「公開承諾」具象化：`Tasks -> Commitments -> Validation` 這三個區塊形成了一套強大的心理學機制。使用者不僅是領取任務，還必須做出承諾，且最終需要社群成員的 Validation（驗證/認可）。這種互相依賴的設計大幅提高了中途放棄的心理成本。
- 社交情境隔離（**Social Context Switching**）：右側的節點導覽列設計非常巧妙。使用者可以根據當下的需求切換社交範圍。在 `Study Group` 裡尋求專業知識的探討，在 `Friend Group` 中獲得情緒支持，甚至在 `Total Strangers` 中進行無壓力的提問或實驗（例如探討虛擬互動的邊界）。這確保了不同屬性的資訊不會互相污染。

三、延伸技術建構需求

要實作這個高度互動且具備即時性的社群面板，需要以下技術的支援：

1. 前端開發技術 **(Frontend Architecture)**

- 版面配置：此種不規則但對齊的區塊，強烈依賴 CSS `Grid Layout` 系統（如 Tailwind 的 `grid-cols-*` 系統），確保在不同解析度下各區塊能平穩縮放。
- 即時資料更新 **(Real-time Updates)**：`Posts / Achievements` 動態牆需要即時性。前端應使用 **WebSocket** 或 **Server-Sent Events (SSE)** 來訂閱伺服器的推播，讓其他人的成就能立刻彈出，增強社群活絡感。
- 狀態管理：當使用者點選右側的子社群（如從 `Friend Group` 切換到 `Study Group`）時，前端狀態管理器需要立刻通知所有版塊（Goals, Posts, Challenges, Tasks）重新抓取對應社群的資料。

2. 後端與資料庫技術 **(Backend & Database)**

- 權限與存取控制 **(RBAC / ABAC)**：這是此頁面最複雜的後端邏輯。資料庫必須嚴格管理「誰屬於哪個社群（Group）」以及「這則貼文的能見度設定」。在 `Total Strangers` 發佈的內容，存取權限邏輯將完全不同於 `Friend Group`。
- 關聯式資料庫設計 **(RDBMS)**：強烈建議使用 PostgreSQL 或 MySQL 來處理複雜的關聯：`Users` 多對多關聯 `Groups`，`Groups` 一對多關聯 `Posts`、`Challenges` 與 `Tasks`。
- 工作流狀態機 **(State Machine)**：針對右下角的機制，後端需要為 Task 設計嚴格的狀態流轉：`Unclaimed` -> `Claimed (Commitment made)` -> `Pending Validation` -> `Validated/Completed`。

3. 系統連動與自動化 **(Integration)**

- 自動推播機制：此頁面不應只依賴使用者手動發文。後端需要設計一套 Event Emitter（事件發布機制）。例如：當使用者在「Daily Report」模組中連續完成 7 天任務時，系統自動捕捉該事件，並在 `Posts / Achievements` 區塊生成一則自動貼文。

5. 對話系統專家主頁（**AI** 幫手**+**配對的專家**(AI**扮演固定個性人設**)**輔助對話系統）

這是系統處理「輸入記錄便利性」與「低心智成本」的核心功能，

有兩種介面設計版本:

版本一:



這是一張聚焦於「AI 幫手（AI Helper）」模組中「專案脈絡與進度檢視（Project Context & Timeline）」的 UI/UX 線框圖。此頁面設計將 AI 輔助從單純的聊天框，昇華為具備特定領域知識、且與使用者長期目標綁定的「多角色協作空間」。

以下為您將此頁面的架構、內容、UI/UX 邏輯，以及實作所需的技術進行完整拆解描述：

一、頁面架構與內容描述

此頁面採用「左側導覽與控制、右側內容與脈絡」的經典雙欄位（Two-Column Split）佈局。

1. 左側邊欄：**AI** 角色與專家陣列 **(AI Persona Sidebar)**

- 全域標題：頂部標示 AI 幫手，旁邊帶有一個經典的機器人/AI 圖示。
- 角色清單：下方列出四位具備專屬人設與專業領域的 AI 角色，每位皆附有頭像佔位符、角色定位與姓名：
    1. 家庭諮商師 - 品安

    2. 動力導師 - Robert
    3. 情感諮商師 - Beth
    4. 焦慮治療師 - 佑棋
- 操作按鈕：側邊欄最下方有一個獨立的 `配對` 按鈕。

2. 右半部上段：專案脈絡設定區 **(Project Context Area)**

此區塊用於定義當前對話或任務的核心語境。

- **`project`** **(**專案名稱**)**：帶有文件圖示，目前設定為 內在動機探索。
- **`intention`** **(**意圖/初衷**)**：明確寫出 找尋探索方法，讓生活更有目標。
- **`Summary`** **(**摘要**)**：預留的空白區塊，預期用於放置目前進度的系統總結。
- **`note`** **(**筆記**)**：預留的空白區塊，供使用者或系統記錄關鍵重點。

3. 右半部下段：里程碑時間軸 **(Milestone Timeline)**

- 起點與終點：一條水平線，左側起點標示為 `Event`（事件/起點），帶有旗幟圖示；右側終點標示為 `Milestone`（里程碑），同樣帶有旗幟圖示。
- 進度節點：時間軸上有數個圓點節點，部分節點下方帶有文件/筆記圖示，代表該時間點有產生特定產出或對話紀錄。其中一個節點上方標註了具體日期 `05-18`。
- 控制元件：右上角有 `+` `-` 符號，預期是時間軸的縮放（Zoom in/out）或新增/刪除節點的功能。

二、UI / UX 互動設計邏輯

此介面的設計深度融合了心理學與目標管理工具的 UX 原則：

- 專屬人設隔離（**Persona Segregation**）：透過切換左側不同的諮商師或導師，使用者能在心理上建立不同的溝通預期（例如：找 Robert 是為了被推動，找 Beth 是為了獲得傾聽）。這種隔離能避免單一萬能 AI 帶來的角色混亂。
- 脈絡錨定（**Context Anchoring**）：右側上方的 `project` 與 `intention` 是這個介面的核心。它強迫系統與使用者在對話前「對齊目標」，避免 AI 的回答發散，確保每一次的互動都是為了解決「內在動機探索」這個核心命題。
- 進度具象化與微型回饋：下方的水平時間軸將漫長的心理探索或專案過程具象化。每經過一次對話或完成一個小任務，時間軸上就多出一個帶有文件的節點，這呼應了您先前構思中「看見進步、降低心智成本、刺激成就感」的設計邏輯。
- 「配對」機制的巧思：左下的 `配對` 按鈕暗示了一種動態路由邏輯。使用者可能不需要自己選擇要找哪位導師，而是輸入當下的心情或困境後，系統自動「配對」最適合接手的 AI 角色。

三、延伸技術建構需求

要將此具備多重角色與長期記憶的 AI 輔助系統實作出來，需要整合前端視覺與後端進階的 AI 代理架構：

1. 前端開發技術 **(Frontend Architecture)**

- 版面與狀態管理：使用 React 或 Vue 進行組件化開發。左側的選擇器與右側的內容高度

連動，需要精準的全域狀態管理（如 Zustand 或 Pinia），確保切換 AI 角色時，右側的對話紀錄或專案脈絡能瞬間切換。
- 時間軸渲染技術：水平時間軸若要支援 `+ -` 的平滑縮放與節點的動態生成，建議不使用純 CSS，而是導入資料視覺化套件（如 **D3.js** 或 **Vis.js Timeline**），以處理複雜的時間區間計算與互動拖曳。

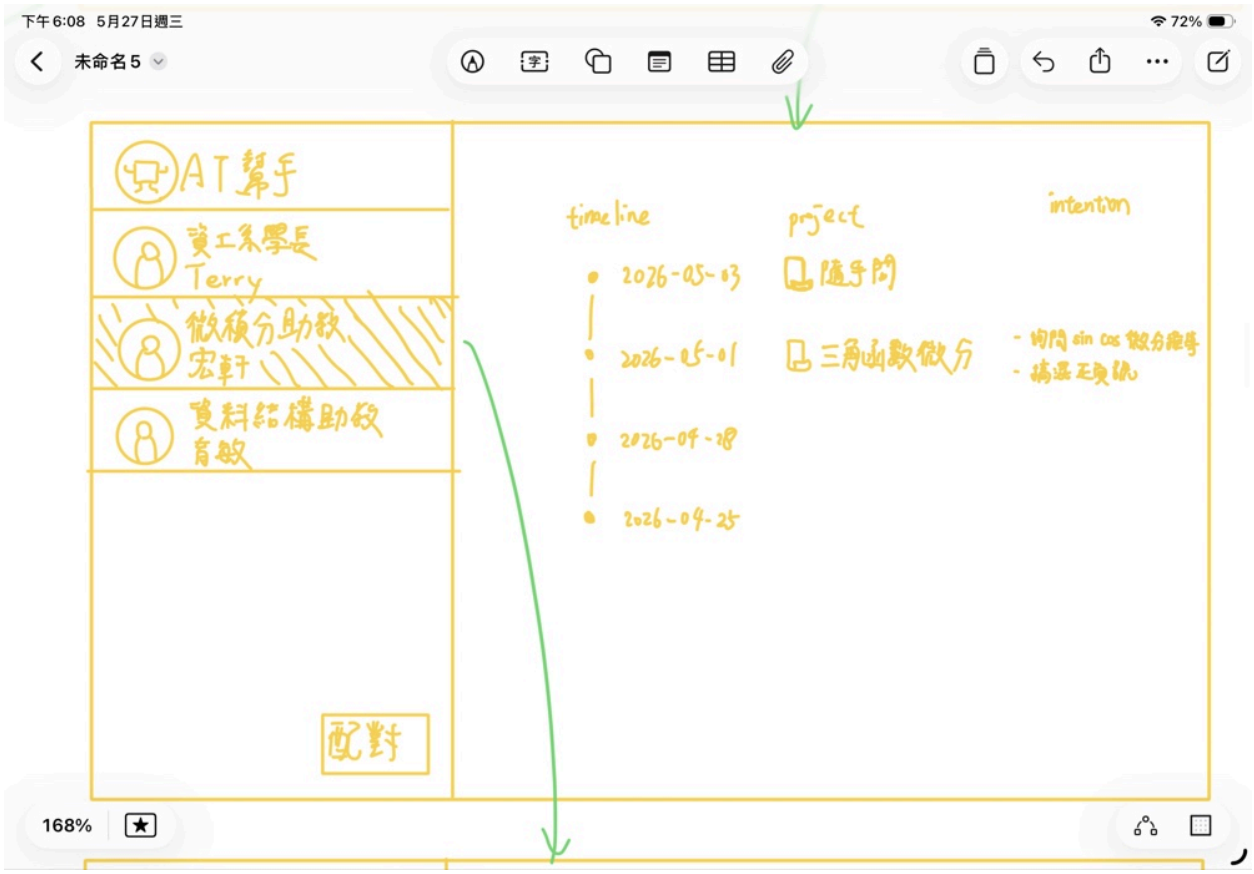
2. 後端與 **AI** 多智能體架構 **(Backend & Multi-Agent Systems)**

- 多智能體協作框架 **(Multi-Agent Framework)**：左側的不同角色極度適合使用 **LangGraph** 這類具備狀態機特性的多智能體框架來建構。
    - 品安、Robert、Beth 各自是一個具備獨立 System Prompt（定義其專業語氣與治療流派）的 Agent 節點。
    - `配對` 按鈕的背後，實作一個「路由代理（Router Agent）」，它會分析使用者的 `intention`，然後將對話流轉發給最適合的專屬 Agent。
- 長期記憶與檢索增強 **(RAG & Memory)**：
    - AI 必須能記住時間軸上的過去事件。需要整合向量資料庫（Vector DB，如 Qdrant 或 Chroma）來儲存過往對話。
    - 右側的 `Summary` 區塊，可由系統背景執行一個「總結 Agent」，在每次對話結束後，自動收斂重點並更新這個欄位。

3. 個人知識管理系統連動 **(PKM Integration)**

- 雙向資料同步：頁面中的 `note` 與 `project` 資訊，在後端可以轉換為 Markdown 格式。透過建立 API 或 Webhook，將這些 AI 整理出的筆記與摘要，利用 Git 版本控制自動推播（Commit）到您個人的 Obsidian 知識庫中，達成無縫的「Life OS」資料流轉，降低手動整理的摩擦力。

版本二:



這是一張延續「AI 幫手（AI Helper）」模組的 UI/UX 線框圖，但情境從上一張的「心理諮商與動機探索」，切換到了「學科輔助與專業知識解答」。此頁面展示了使用者如何檢視特定 AI 角色（如學科助教）的過往互動歷史。

以下為您將此頁面的架構、內容、UI/UX 邏輯，以及實作所需的技術進行完整拆解描述：

一、頁面架構與內容描述

此頁面同樣採用雙欄位（Two-Column Split）佈局，但右側的內容展示邏輯與前一頁截然不同，從「單一專案的水平進度」變成了「特定角色的垂直互動歷史」。

1. 左側邊欄：學科與專業 **AI** 角色陣列 **(Academic Persona Sidebar)**

- 全域標題：頂部標示 AI 幫手，旁邊帶有機器人圖示。
- 角色清單：列出針對不同學科或情境設定的 AI 角色：
    1. 資工系學長 - Terry
    2. 微積分助教 - 宏軒（此區塊帶有斜線陰影，表示目前為選取/啟動狀態）
    3. 資料結構助教 - 育敏
- 操作按鈕：底部保留了 配對 按鈕。

2. 右半部：特定角色的歷史互動時間軸 **(Role-Specific History Timeline)**

此區塊呈現了與目前選取角色（微積分助教）的過往對話或任務紀錄。

- 欄位標題：頂部由左至右排列三個欄位指標：`timeline`（時間軸）、`project`（專案/主題）、`intention`（意圖/發問目的）。
- 時間軸內容（由近到遠垂直排列）：
  - 節點 **1 (`2026-05-03`)**：`project` 為 `隨手問`（帶有文件圖示），`intention` 留白。
  - 節點 **2 (`2026-05-01`)**：`project` 為 `三角函數微分`，`intention` 明確列出兩點：`- 詢問 sin cos 微分推導` 與 `- 搞懂正負號`。
  - 節點 **3 & 4 (`2026-04-28`, `2026-04-25`)**：僅標示時間節點，作為過往歷史的延伸視覺呈現。

二、UI / UX 互動設計邏輯

此介面的核心體驗在於「領域知識的隔離」與「學習脈絡的延續」。

- 視覺狀態的明確提示（**Active State Indicator**）：左側「微積分助教宏軒」區塊的斜線陰影，是一個強烈的視覺回饋，讓使用者瞬間明白右側顯示的所有歷史紀錄，都是專屬於微積分領域的。綠色箭頭也暗示了從這個選取狀態，可以向下進入更深層的具體對話畫面。
- 垂直時間軸的梳理（**Chronological Context**）：將過去散落的「聊天紀錄」結構化為「Timeline -> Project -> Intention」。這解決了傳統 LLM 聊天視窗中「找不到過去問過的問題」的痛點。
- 意圖與主題的前置化（**Intention-Driven Log**）：在列表中直接顯示發問意圖（如：搞懂正負號），幫助大腦快速回憶當時的學習卡關點。使用者點擊該節點後，就能無縫接軌當時的思維邏輯，繼續深入探討，大幅降低重新進入學習心流的摩擦力。

三、延伸技術建構需求

要實作這個具備學科隔離與歷史檢索能力的介面，需要以下前端渲染與後端 AI 代理技術的支援：

1. 前端開發技術 (Frontend Architecture)

- 佈局與條件渲染：使用 React 或 Vue。右側的時間軸必須是依賴左側選取狀態（Active Role）的子組件。當左側狀態從「微積分助教」切換到「資工系學長」或「資料結構助教」時，右側必須立即觸發資料重新抓取（Data Fetching），渲染對應的程式碼或課業討論歷史。
- 垂直時間軸實作：有別於上一張圖的水平時間軸，這裡的垂直時間軸相對單純。可使用 CSS 的 `border-left` 搭配相對定位的圓點（`::before` 偽元素）來繪製貫穿的垂直線與時間節點。
- 動態列表與分頁 **(Pagination/Infinite Scroll)**：如果歷史紀錄很多，右側的清單需要實作無限捲動或分頁載入，避免一次拉取過多資料導致效能瓶頸。

2. 後端與多智能體架構 (Backend & Multi-Agent Systems)

- 角色隔離的 **Prompt** 工程與 **LangGraph** 架構：
  - 針對「資工系學長」、「微積分助教」這類高度專業的角色，非常適合透過 LangGraph 建立獨立的 Agent 節點。
  - 每個助教 Agent 都需要綁定專屬的 System Prompt 與工具。例如：微積分助教需

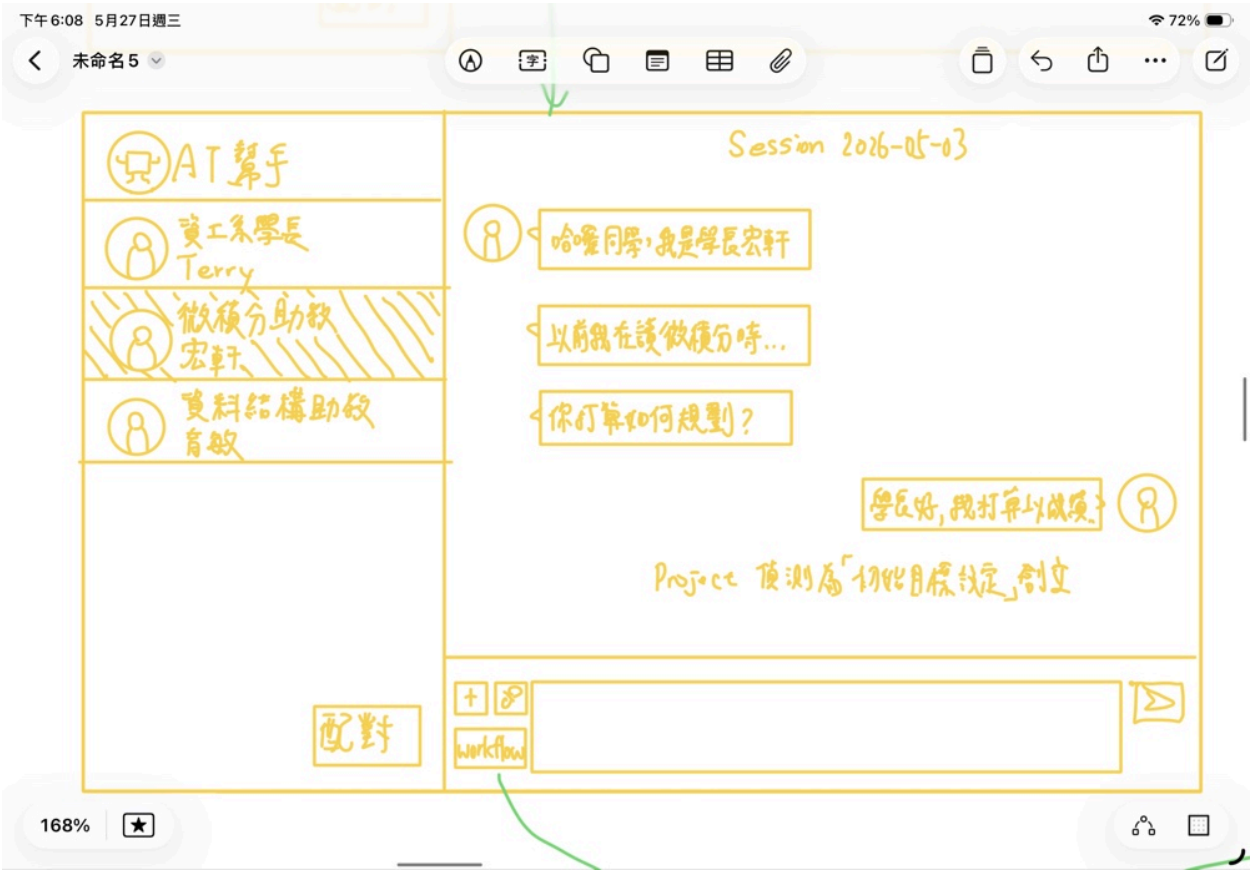
要被指示強制使用 LaTeX 語法輸出數學公式（如 `$\frac{d}{dx} \sin(x) = \cos(x)$`）；資料結構助教則可能需要具備程式碼執行或演算法視覺化的擴充工具。

- 記憶萃取與結構化儲存 **(Memory Extraction)**：
  - 要在列表中精準顯示 `project` 與 `intention`，不能依賴人工輸入。系統需要在每次對話結束後（Session End），利用一個背景執行的「總結 Agent」去閱讀整段對話，自動萃取出「這次討論的主題（Project）」與「使用者的核心痛點（Intention）」，並將其結構化寫入關聯式資料庫中。

3. 資料庫設計 **(Database Design)**

- 實體關聯：需要建立 `Users` -> `Roles` (AI Personas) -> `Sessions` (對話階段) 的關聯。
- 右側顯示的每一列，實際上就是一筆 `Session` 資料，包含欄位：`session_id`, `role_id`, `created_at` (Timeline), `title` (Project), `extracted_intentions` (JSON array)。當點擊特定節點時，再透過 `session_id` 從向量資料庫或文件資料庫中撈取完整的對話紀錄。

6. 對話系統（對話介面）



這是一張展示「AI 幫手（AI Helper）」模組最深層級——「實際對話與互動介面（Session View）」的 UI/UX 線框圖。畫面從前一張的歷史時間軸，正式進入了與特定 AI 角色即時對談的情境。

以下為您將此頁面的架構、內容、UI/UX 邏輯，以及實作所需的技術進行完整拆解描述：

一、頁面架構與內容描述

此頁面延續了雙欄位（Two-Column Split）佈局，左側保持導覽列狀態，右側則切換為標準的通訊軟體（Chat UI）介面。

1. 左側邊欄：**AI** 角色導覽區 **(Persona Sidebar)**

- 狀態延續：與上一張圖相同，列出了針對資工領域設定的 AI 角色：`資工系學長 Terry`、`微積分助教 宏軒`、`資料結構助教 育敏`。
- 當前啟用狀態：`微積分助教 宏軒` 的區塊帶有斜線陰影，表示使用者目前正在與這位角色進行 Session（對話階段）。底部同樣保留 `配對` 按鈕。

2. 右半部：對話互動區 **(Session Chat Interface)**

- 頂部資訊 **(Header)**：標示了當前的對話階段與日期 `Session 2026-05-03`。
- 對話內容流 **(Message Feed)**：
  - **AI** 角色發言（靠左，帶有頭像）：
    - 「`哈囉同學，我是學長宏軒`」
    - 「`以前我在讀微積分時...`」
    - 「`你打算如何規劃？`」
  - 使用者發言（靠右，帶有頭像）：
    - 「`學長好，我打算以成果...`」
  - 系統自動事件（置中提示文字）：在對話下方出現了一行置中的系統通知：「`Project 偵測為「初始目標設定」創立`」。
- 底部輸入區 **(Input Bar)**：
  - 左側功能鍵：包含 `+`（新增附件/選單）、🔗（插入連結）以及一個特別標示為 `workflow` 的按鈕（帶有一條向外延伸的線線，暗示可展開或觸發特定流程）。
  - 文字輸入框：中央為空白的輸入區域。
  - 右側送出鍵：帶有紙飛機或箭頭圖示 ▷ 的發送按鈕。

二、UI / UX 互動設計邏輯

此對話介面不僅僅是一個簡單的聊天室，它隱含了多項高階的對話式體驗設計（Conversational UX）：

- 擬真化的碎塊發言（**Chunked Messaging**）：AI 的開場白被拆分成了三個獨立的對話泡泡（Bubble），而不是長篇大論的一整段文字。這種設計完美模仿了真人用通訊軟體打字的節奏，降低了閱讀壓迫感，也更符合「學長」這種親近的人設。
- 引導式破冰與同理對話：AI 主動出擊，不僅自我介紹，還透過「以前我在讀微積分時...」來展現同理心，最後用「你打算如何規劃？」將發言權交回給使用者。這有效解決了使用者面對空白對話框時的「冷啟動（Cold Start）」焦慮。
- 隱形的自動化（**Invisible Automation**）：置中的那句 `Project 偵測為「初始目標設定」創立` 是整個畫面的神來之筆。它代表系統在背後偷偷「聽懂了」對話脈絡，並自動幫使用者建立好專案與目標追蹤節點。使用者不需要跳出對話框去手動填寫表單，完美實現了前面一直強調的「低心智成本高品質產出」。
- 進階工具的入口（**Workflow Trigger**）：輸入框旁的 `workflow` 按鈕，暗示了這個對話框同

時也是一個「指令列介面（CLI）」。使用者可以隨時叫出自動化工作流或範本，快速串接其他的開發或整理任務。

三、延伸技術建構需求

要打造這個高度擬真且具備背景處理能力的對話介面，需要整合前端即時渲染、後端串流，以及多智能體（Multi-Agent）架構技術：

1. 前端開發技術 **(Frontend Architecture)**

- 即時對話介面實作：對話區塊需要處理複雜的狀態，包含使用者的輸入、AI 的打字特效（Typing Indicator）、以及不同訊息類型（Message、System Event）的條件渲染。這通常依賴 React/Vue 來管理 Component 狀態，並需要精確處理自動捲動到底部（Auto-scroll to bottom）的 UX 細節。
- 串流傳輸 **(Streaming Response)**：為了讓 AI 的回覆有「逐字吐出」的真實感，前端需要實作 Server-Sent Events (SSE) 或 WebSocket 來接收後端 LLM 傳回的串流字元。

2. 後端與多智能體協作框架 **(Backend & Agent Framework)**

這個介面非常適合使用 **LangGraph** 等框架來實作複雜的後端邏輯：

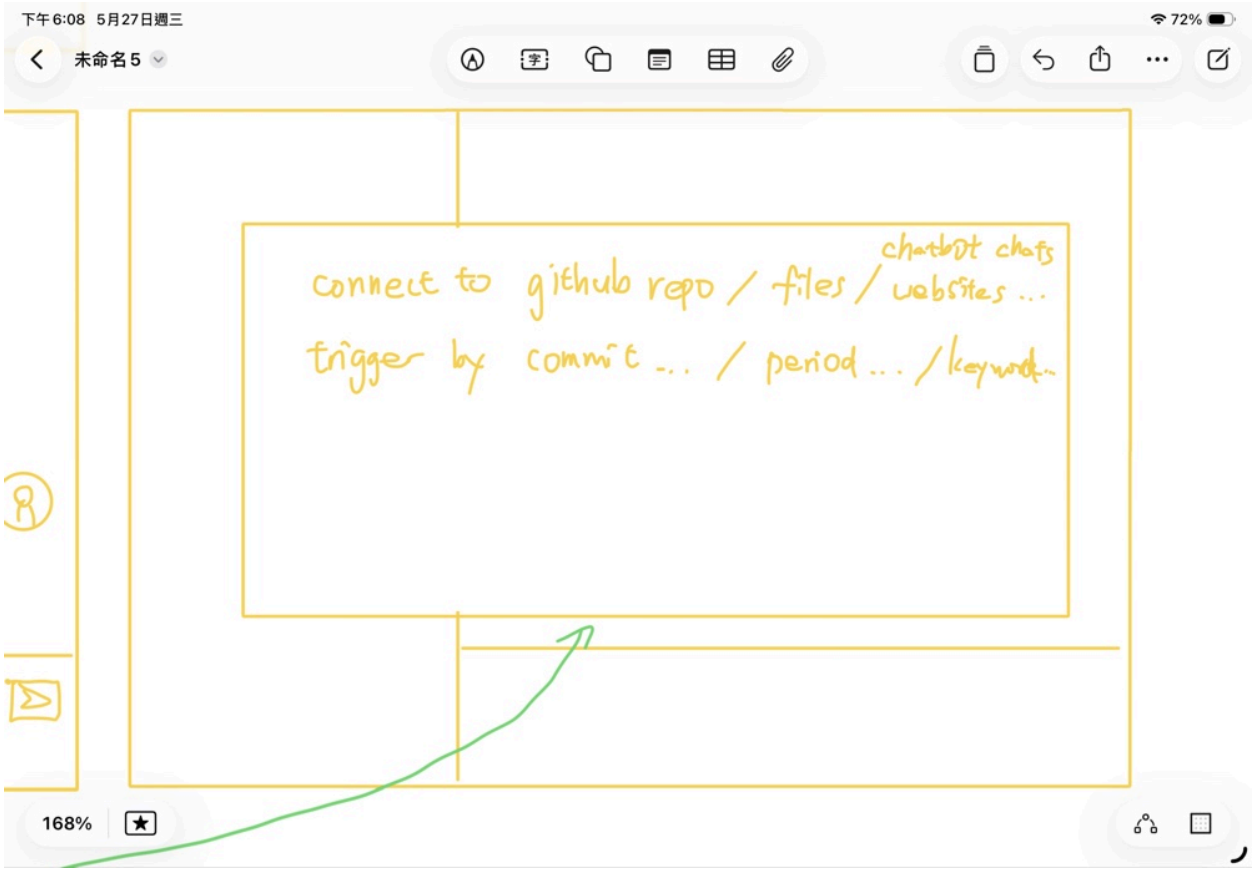
- 角色塑造與系統提示詞 **(Persona Prompting)**：「學長宏軒」這個 Agent 節點必須被注入非常具體的 System Prompt。例如：「你是一位就讀資工系的學長，正在輔導學弟妹微積分。說話語氣要輕鬆、親切，並且常分享自己過去的學習經驗。每次回覆請控制在 *3* 句話以內，並多用問句引導。」
- 意圖偵測與背景工具調用 **(Intent Extraction & Tool Calling)**：要實現 `Project 偵測為「初始目標設定」創立` 這樣的功能，系統在處理對話時不能只有一個 Agent。
  - 當前台 Agent 在跟使用者聊天的同時，後台必須有一個平行的「觀察者 Agent（Observer/Extractor）」在傾聽對話。
  - 一旦觀察者 Agent 發現使用者提及了「打算如何做...」，它會自動觸發 **Function Calling**（函數調用）。
  - 系統在背景執行建立資料庫關聯的操作（寫入 Project 表單），完成後將結果透過 System Notification 格式傳回前端，顯示在對話流中，達成無縫的自動化整理。

3. Workflow 系統整合

- 左下角的 `workflow` 按鈕，意味著系統支援自訂腳本。這裡可以設計成讓使用者自行編寫或串接外部服務。例如：觸發一個 Python 爬蟲腳本去抓取微積分考古題，或是呼叫 API 自動將剛才談論的「初始目標」同步打包成一份 Markdown 檔案，自動 Commit 上傳並整合到個人的知識庫中。

7. 第三方系統連動與觸發（右下角）

綠色箭頭將 AI 對話的輸入框以及社群頁面的動作，指向了這個自動化區塊。



這是一張展示「工作流與自動化觸發（Workflow & Automation Triggers）」設定面板的 UI/UX 線框圖。從畫面下方的綠色箭頭走向可以判斷，這正是由上一張對話介面中輸入框旁邊的 `workflow` 按鈕所觸發的進階設定彈出視窗（Modal / Overlay）。

以下為您將此頁面的架構、內容、UI/UX 邏輯，以及實作所需的技術進行完整拆解描述：

一、頁面架構與內容描述

此頁面跳脫了底層的雙欄位排版，採用了置中的懸浮視窗（Modal/Dialog）架構，用於處理系統底層的資料串接與自動化規則設定。

1. 視覺層次與連動關係 (Visual Hierarchy & Flow)

- 底層背景：隱約可見底層的側邊欄與對話框輸入區，表示這是一個不脫離當前對話脈絡的覆蓋層。
- 觸發路徑（綠色箭頭）：明確標示了這個面板是由下方輸入區的按鈕向上展開，確立了操作的因果關係。

2. 設定面板內容 (Configuration Panel)

面板內分為兩大核心設定區塊：

- 資料來源綁定 **(Data Sources / Inputs)**：

    - `connect to github repo / files / websites...`(連接至 GitHub 儲存庫 / 檔案 / 網站...)
    - 右上角有小字補充 `chatbot chats`(聊天機器人的歷史對話)。
    - 此區塊定義了 AI 與系統可以去哪裡「抓取」使用者的行為軌跡與外部知識。
- 自動化觸發條件 **(Triggers / Events)**:
    - `trigger by commit... / period... / keyword...`(透過 Git Commit 紀錄 / 特定週期時間 / 特定關鍵字... 來觸發)。
    - 此區塊定義了系統「何時」應該主動執行工作流。

二、UI / UX 互動設計邏輯

此介面是整個軟體能達到「低心智成本」的技術核心，其 UX 設計邏輯著重於「將主動記錄轉化為被動監聽」:

- 無感記錄(**Zero-Friction Logging**):使用者不再需要每天打開「日報」手動輸入「我今天寫了什麼程式碼」。透過設定 `connect to github repo` 並 `trigger by commit`，只要在終端機推播程式碼，系統就會自動將 Commit Message 轉換為經驗值(XP)與專案進度。
- 脈絡內設定(**In-Context Configuration**):將這個設定介面做成由對話框直接喚出的 Modal，而不是藏在深層的「系統設定」頁面中。這讓使用者能在與 AI 討論到一半，意識到需要自動化時，就能順手設定好一個爬蟲任務或定時提醒。
- 知識庫無縫掛載:支援連接 `files`(如本地的 Markdown 筆記庫)與 `websites`。這意味著 AI 幫手不只是個聊天的語言模型，它能隨時存取最新的個人知識節點與外部文獻，作為後續對話的增強依據。

三、延伸技術建構需求

這個看似簡單的設定面板，背後需要一套強大且非同步的後端架構與事件驅動機制來支撐:

1. 前端開發技術 **(Frontend Architecture)**

- 彈出視窗實作 **(Modal / Portals)**:使用 React Portals 或 Vue Teleport 技術。雖然按鈕在底層組件，但渲染時將此 Modal 提升至 DOM 樹的頂層，避免受到父層 CSS(如 `overflow: hidden` 或 `z-index`)的干擾。
- 表單與驗證狀態:需要實作動態表單(Dynamic Forms)。當使用者選擇不同的 Trigger(如 `period` vs `commit`)，下方需要連動渲染不同的設定欄位(如 Cron Job 語法輸入框 vs GitHub Repo 選擇器)。

2. 後端與自動化架構 **(Backend & Automation Systems)**

- **Webhook** 接收器 **(Webhook Receiver)**:
    - 為了實現 `trigger by commit`，後端需要開放特定的 API 端點(Endpoints)來接收來自 GitHub 的 Webhook 請求(Payload)。當接收到 `push` 事件時，解析 JSON 資料並觸發對應的工作流。
- **OAuth 2.0** 授權機制:
    - `connect to github repo` 需要實作完整的 OAuth 流程，安全地獲取使用者的 Access Token，以便系統能合法讀取使用者的 Private Repositories。
- 排程任務系統 **(Task Scheduling)**:

    - 針對 `trigger by period...`，後端需要整合排程套件。在 Node.js 中可使用 `node-cron` 或 `BullMQ`；若架構在雲端（如 AWS），則可利用 EventBridge 來觸發定時的 Lambda 函數執行。
- 事件驅動架構 **(Event-Driven Architecture, EDA)**：
    - 系統底層建議導入訊息佇列（Message Queue，如 Redis Pub/Sub、RabbitMQ）。當任何觸發器（Trigger）被啟動時，先將事件推入佇列，再由背景的 Worker 節點非同步處理（例如：呼叫 LLM 總結程式碼、更新資料庫進度），避免阻塞主伺服器效能。

3. 檢索增強生成架構 **(RAG Pipeline)**

- 針對 `connect to files / websites`，這牽涉到資料攝取（Data Ingestion）。系統需要具備解析不同格式（Markdown, PDF, HTML）的能力，將文本進行切塊（Chunking），轉換為向量（Embeddings），並儲存至向量資料庫中，讓 AI 代理（Agents）在未來的對話中能透過相似度檢索調用這些外部知識。

整體架構分析總結

這套產品架構的動線非常清晰：透過底層的 自動化觸發（**GitHub Webhooks**）與 **AI** 幫手的對話（降低輸入成本）作為資料收集的入口；資料進入系統後，會被分類到特定的 角色空間（**Dashboard**）中進行運算與儲存；接著轉化為個人的 進度回顧（**Daily Report**）與 視覺化成就（**Collection**）以刺激多巴胺；最終將這些進度推送到 社群（**Community**）中，與學術或專案夥伴產生連結，利用社會情懷來穩固長期的使用習慣。